

Overview

Reinforcement Learning (RL) is the study of agents that learn behavior by interacting with an environment and optimizing long-term cumulative reward. Unlike supervised learning, RL does not usually receive a labeled “correct action” for each state. Unlike unsupervised learning, RL is explicitly goal-directed through rewards tied to sequential decisions.

Core distinction:

- Supervised learning: learn mapping $x \rightarrow y$ from labeled examples.
- Unsupervised learning: discover structure in unlabeled data.
- RL: learn policy $\pi(a \mid s)$ that maximizes expected return through trial and error.

A generic objective is:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

where τ is a trajectory, r_t rewards, and $\gamma \in [0, 1)$ the discount factor.

Why RL is hard in practice:

- data is non-i.i.d. and distribution shifts as policy changes,
- delayed rewards make credit assignment difficult,
- exploration risks and sample inefficiency are significant,
- training instability appears with function approximation.

Markov Decision Processes (MDPs)

An MDP is the standard mathematical model for RL:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

- $\mathcal{S}$: state space
- $\mathcal{A}$: action space
- $P(s' \mid s, a)$: transition dynamics
- $R(s, a)$: reward function
- γ : discount factor

Policy and Value Functions

A policy π maps states to action probabilities.

- State value:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right]$$

- Action value (Q-function):

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right]$$

- Advantage:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

Bellman Equations

For a fixed policy:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Optimal action value satisfies Bellman optimality:

$$Q^*(s, a) = \sum_{s'} P(s' | s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

Core RL Algorithms

Tabular Methods

Q-learning (Off-policy)

Update rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

- Off-policy: learns greedy target regardless of behavior policy.
- Strong baseline for small/discrete environments.

SARSA (On-policy)

Update rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

- On-policy: updates with action actually chosen under current policy.
- Often more conservative in risky settings.

Policy Gradient Basics

Policy gradients directly optimize $J(\theta)$ for parameterized policy π_θ :

$$\nabla_\theta J(\theta) = \mathbb{E} [\nabla_\theta \log \pi_\theta(a_t | s_t) G_t]$$

- REINFORCE is unbiased but high variance.
- Actor-critic reduces variance via value baseline.

Deep Q-Networks (DQN)

DQN approximates $Q(s, a)$ with neural network Q_θ :

$$L(\theta) = \mathbb{E} [(y_t - Q_\theta(s_t, a_t))^2], \quad y_t = r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a')$$

Key stabilizers:

- target network θ^- ,
- replay buffer for decorrelated samples.

High-level variants:

- Double DQN (reduced overestimation bias),
- Dueling networks (separate value and advantage streams),
- Prioritized replay (sample informative transitions more often).

Exploration vs Exploitation

Exploration selects uncertain actions to gather information; exploitation selects currently best-known actions.

ϵ -greedy

- with probability ϵ , take random action,
- otherwise take $\operatorname{argmax} Q$ action.

Pros: simple. Cons: uninformed random exploration.

Softmax/Boltzmann Exploration

Actions sampled proportional to exponentiated value estimates:

$$P(a \mid s) \propto e^{Q(s,a)/\tau}$$

Temperature τ controls randomness.

Curiosity and Intrinsic Motivation (Intuition)

Agent gets intrinsic reward for novel or uncertain states. Useful in sparse-reward tasks but can misalign with true task objectives if poorly tuned.

Practical Deep RL Considerations

Replay Buffers

Store transitions $(s, a, r, s', done)$ and sample mini-batches for training.

Benefits:

- breaks temporal correlations,
- improves data efficiency through reuse.

Target Networks

A lagged copy of Q-network prevents rapidly shifting targets and improves optimization stability.

Training Instability and Sample Inefficiency

Common issues:

1. unstable bootstrapping with function approximation,
2. sensitivity to reward scaling and hyperparameters,
3. huge interaction budget vs supervised alternatives,
4. poor sim-to-real transfer in robotics/control.

Mitigations:

- reward normalization and clipping,
- conservative learning rates,

- evaluation on separate seeds/environments,
- curriculum/domain randomization for transfer.

Business & Domain Use Cases

Robotics and Control

RL can learn closed-loop policies for locomotion, manipulation, and adaptive control in uncertain environments.

Recommendation and Personalization

Sequential recommendation can be framed as long-term reward optimization rather than one-step click prediction.

Operations and Resource Allocation

Dynamic dispatching, inventory, and scheduling tasks may benefit when actions affect future system state.

Business Case Studies & Exceptions

Case Study 1: RL for Inventory Replenishment

Scenario:

- Retail chain with seasonal demand and stockout penalties.

RL framing:

- state: inventory levels, demand forecast features, lead time,
- action: reorder quantities,
- reward: profit - holding cost - stockout cost.

Why RL helps:

- captures sequential downstream effects of current replenishment action.

Operational challenge:

- offline simulation fidelity and safety constraints determine viability.

Case Study 2: Dynamic Pricing Policy Optimization

Scenario:

- Marketplace sets prices over time under demand uncertainty.

RL upside:

- balances short-term conversion and long-term margin.

Risks:

- strategic user behavior can destabilize learned policy,
- fairness/compliance constraints may limit exploratory pricing.

Exceptions: When RL Is Overkill

- If environment does not materially change with action, supervised learning may be enough.
- If decisions are single-step and independent, contextual bandits are often cheaper and easier.
- If reward is poorly defined or delayed beyond reliable attribution, RL may fail silently.

Interview Questions & Answers

1. **Define an MDP.**
An MDP is a tuple (S, A, P, R, γ) describing states, actions, transition dynamics, rewards, and discounting for sequential decision-making.
2. **What is the objective of RL?**
To learn a policy maximizing expected cumulative discounted reward.
3. **Explain Q-learning update rule.**
It bootstraps Q-value toward immediate reward plus discounted max future Q from next state.
4. **Difference between Q-learning and SARSA?**
Q-learning is off-policy (greedy target), SARSA is on-policy (uses actual next action).
5. **What is exploration vs exploitation?**
Exploration gathers new information; exploitation uses current best-known action.
6. **Why is deep RL unstable?**
Bootstrapping, non-stationary targets, correlated data, and sensitive hyperparameters.
7. **Why use replay buffers?**
To decorrelate samples and reuse past experiences for better sample efficiency.
8. **Why use target networks in DQN?**
To stabilize learning by slowing movement of bootstrapped targets.
9. **What is policy gradient?**
A class of methods that directly optimize policy parameters via gradient ascent on expected return.
10. **When would PPO be preferred in practice?**
For robust policy optimization with stable clipped updates in continuous or complex control.
11. **What is sample inefficiency in RL?**
Needing very many environment interactions to learn useful behavior.
12. **How do you evaluate an RL agent?**
Average return across multiple seeds, robustness tests, and constraint violation rates.
13. **What are sparse rewards?**
Reward signals that occur infrequently, making credit assignment difficult.
14. **When is a bandit better than RL?**
When each action has mostly immediate effects and little long-term state dependence.
15. **How do you move RL from simulation to production?**
Use high-fidelity simulators, domain randomization, conservative rollout, and human/constraint guardrails.

References

- UC Berkeley CS285: <https://www2.eecs.berkeley.edu/Courses/CS285/> and <https://rail.eecs.berkeley.edu/deeprlcourse-fa23/>
- Gymnasium docs: <https://gymnasium.farama.org/> and <https://gymnasium.farama.org/api/env/>

- Stable-Baselines3 DQN/PPO docs: <https://stable-baselines3.readthedocs.io/en/v2.4.1/modules/dqn.html> and <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>
- OpenAI Spinning Up PPO docs: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>
- Sutton & Barto (RL book resource mirror): <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook>